

Chipathon

Analog Guide:

<https://github.com/sscs-ose/sscs-chipathon-2026/blob/main/resources/Analog/eda/README.md>

Local Repo: <https://github.com/SamS581/hungry-chippos-chipathon-2026>

Information

https://en.wikipedia.org/wiki/Successive-approximation_ADC

- Cycles through all bits by turning the nth bit into a 1, resetting to 0 if $V_{in} < V_{cmp}$ else keeping

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10704295>

- Optimize speed, resolution, and power

<https://techschems.com/sar-adc-schematic>

https://pages.jh.edu/aandreo1/216/Archives/2014/Handouts/POP_Ch3-4.pdf

- Logic gates from NFETs and PFETs + Rules

Example Designs

<https://www.yilectronics.com/Students/tsnakai/Labs/Project/Project.html>

- Make NAND → make D flip flop → 8-bit SAR
- Tested at each step

https://www.ee.columbia.edu/~kinget/EE6350_S16/03_ADC_Dhruv_Gurkaran_Shikhar/schematic.html

- 16 flip flops (8 shift, 8 control)

<https://gemini.google.com/share/b2b6cdf181c7>

<https://share.google/aimode/s1aaorTjUL1QYcPYM>

- Option 1: xschem with multiple shift register+ring counters
- Option 2: Verilog with coded block

https://www.yilectronics.com/Courses/ENGR338_CE/f2021/proj/SARADC_LTSpiceReport.pdf

Design

<https://notebooklm.google.com/notebook/a845c4f2-c32e-43c8-9e55-b57e1f3be735?authuser=2>

Explanation (Aled cause I didn't have time)

1. Inverter (INV)

- **Structure:** A standard CMOS inverter consists of a single **PMOS transistor on top** (pull-up) and a single **NMOS transistor on the bottom** (pull-down). Their drains are connected to form the output, and their gates are tied together for the input. In your design, the PMOS is twice as wide as the NMOS to balance their switching speeds.
- **Function:** It is the most basic logic element. It simply **flips the logic state**: if the input is High (3.3V), the output is forced Low (0V), and vice versa.

2. Transmission Gate (TG)

- **Structure:** A TG is built by parallel-connecting an **NMOS and a PMOS transistor**. Unlike standard gates, the input and output are interchangeable, making it a bidirectional switch.
- **Function:** It acts as an **ideal electronic switch**. Because it uses both transistor types, it can pass the full \$0V\$ to \$3.3V\$ signal range without the signal loss (threshold voltage drop) that happens with a single transistor. It requires complementary control signals (\$CLK\$ and \$CLK_B\$) to turn both transistors on or off simultaneously.

3. NAND2 and NOR2 Gates

- **NAND2 Structure:** This gate uses two PMOS transistors in **parallel** for the top half and two NMOS transistors in **series** for the bottom half.
- **NAND2 Function:** It outputs a Logic '0' **only** when both inputs are High [Conversation History].
- **NOR2 Structure:** This is the opposite of the NAND; it uses two PMOS transistors in **series** on top and two NMOS transistors in **parallel** on the bottom.
- **NOR2 Function:** It outputs a Logic '1' **only** when both inputs are Low [Conversation History].

4. DFF with Active-Low Asynchronous Reset (DFF_RST_N)

- **Structure:** This is a **master-slave architecture**. It uses two internal storage stages: the "Master" captures data while the clock is low, and the "Slave" captures the master's state and sends it to the output \$Q\$ on the rising edge of the clock. To implement the reset, the standard feedback inverters are replaced with **NAND2 gates**.
- **Function:** It "remembers" one bit of data. The **Active-Low Asynchronous Reset** (**RST_N**) means that if the reset pin hits \$0V\$, the flip-flop immediately clears its output to '0' [137, Conversation History].

5. DFF with Active-Low Asynchronous Preset (DFF_SET_N)

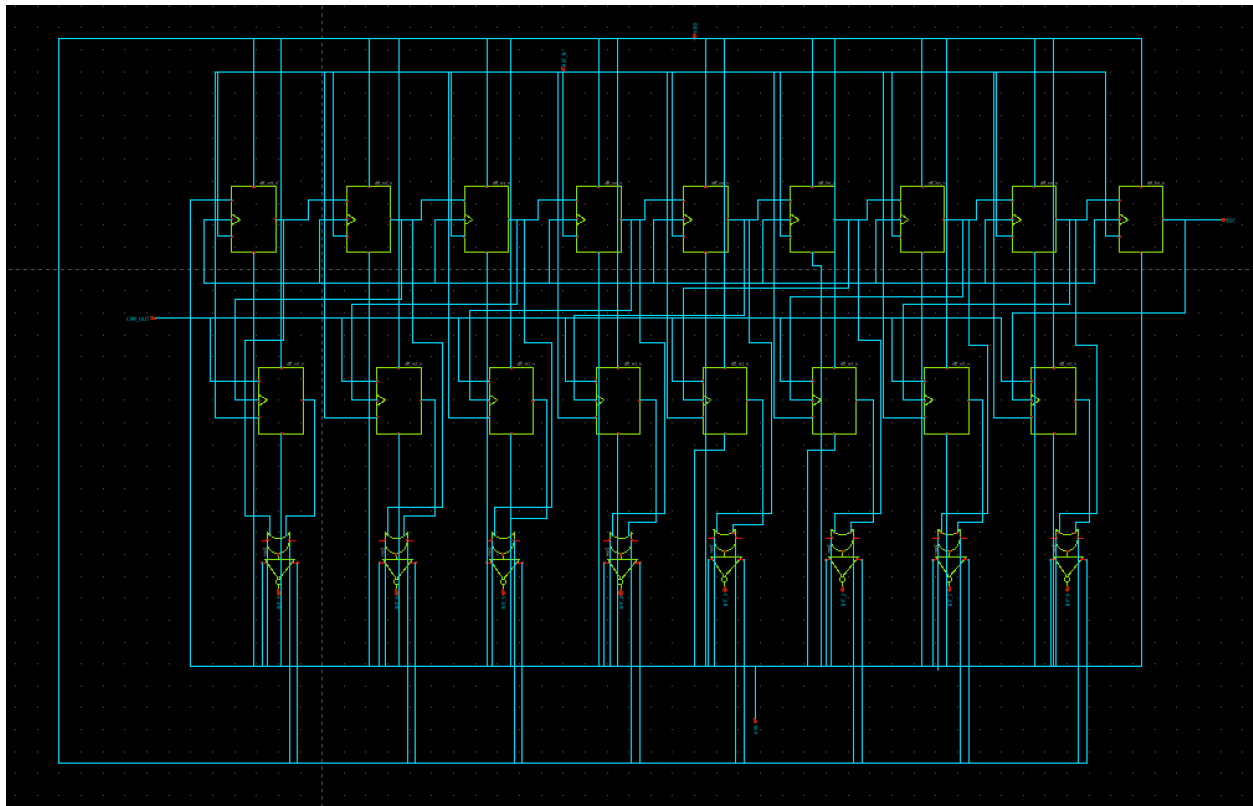
- **Structure:** This variant is nearly identical to the reset version but uses **NOR2 gates** in its feedback loops instead of NANDs. It also includes an extra inverter to translate the active-low reset signal into an active-high trigger required by the NOR gates [140, Conversation History].
- **Function:** It initializes the output to **Logic '1'** immediately when reset is active. This is used exclusively for the very first bit of your ADC's search sequence.

6. Top-Level SAR Logic (SAR_LOGIC)

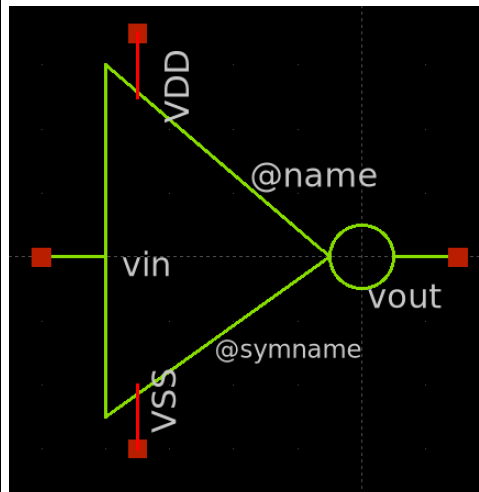
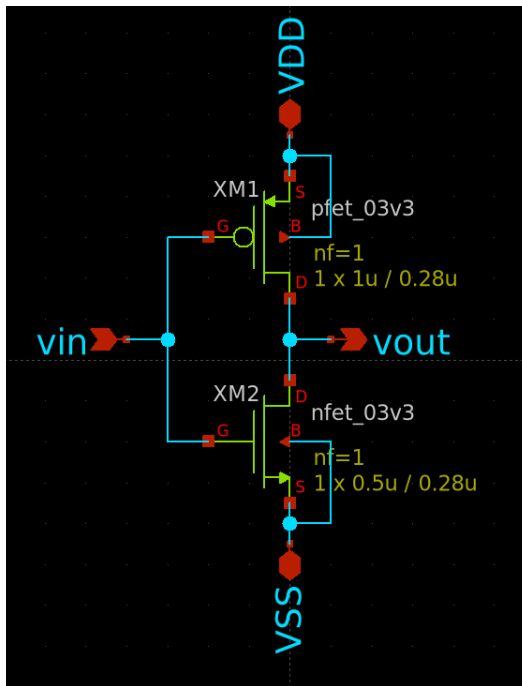
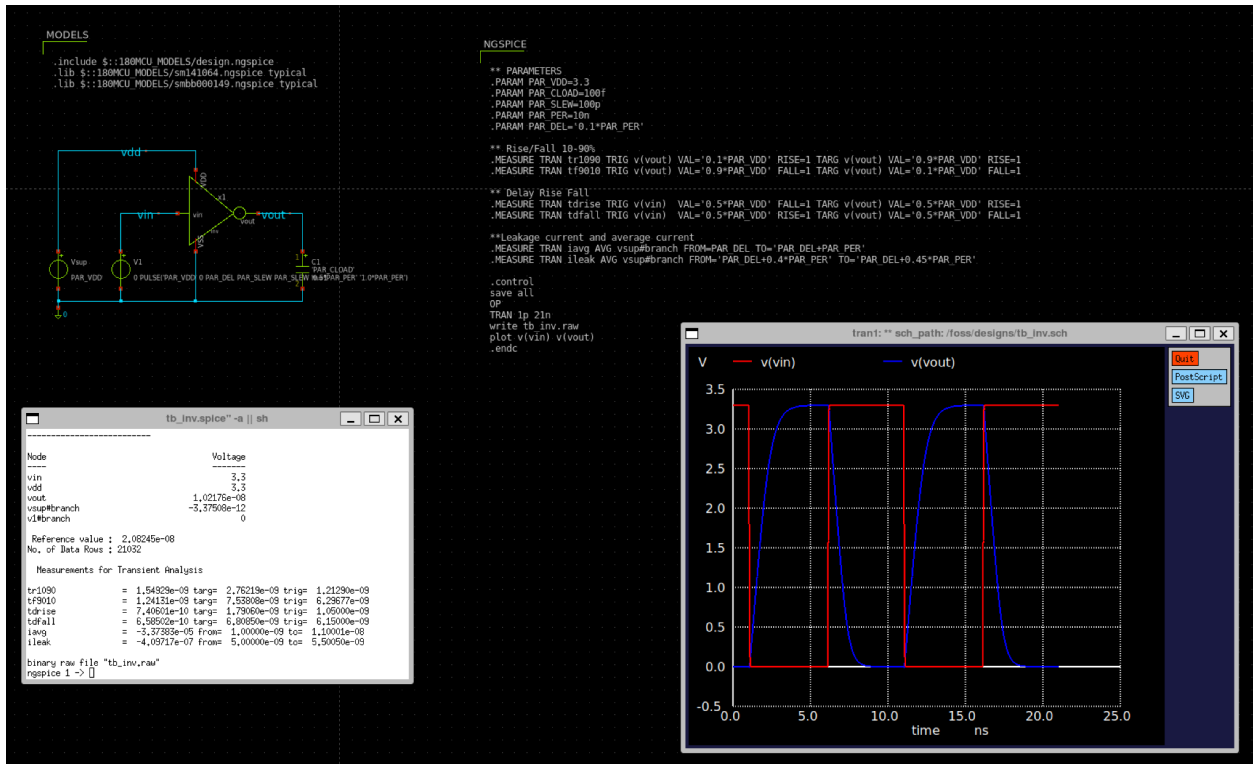
- **Structure:** This integrated block follows the **Anderson architecture**, organizing **17 flip-flops** into two distinct rows:
 1. **Sequencer (Top Row):** 9 flip-flops (one **DFF_SET_N** followed by eight **DFF_RST_N**).

2. **Code Register (Middle Row):** 8 `DFF_RST_N` units.
3. **DAC Drive Logic (Bottom Row):** 8 OR gates (each made of a **NOR2 + INV**) [127, 144, Conversation History].
- **Function:** This is the "brain" of the ADC. It performs the **binary search algorithm**:
 1. The **Sequencer** tracks the progress, moving a single "1" from bit to bit like a pointer.
 2. The **Code Register** captures the comparator's decisions at each step and holds them.
 3. The **OR gates** combine these two rows to tell the Capacitor DAC exactly which switches to flip during the conversion.
 4. Finally, it signals **End-of-Conversion (EOC)** once all 8 bits are resolved.

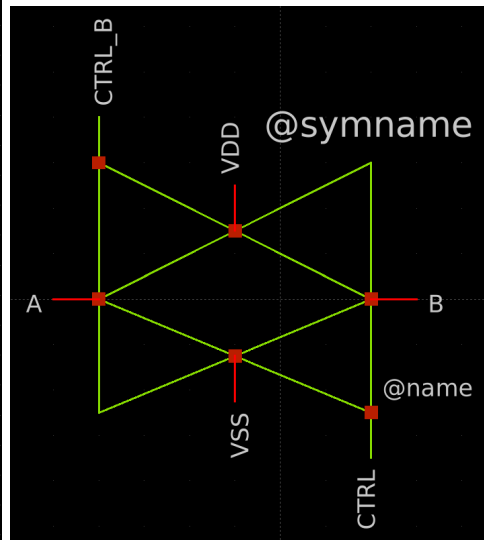
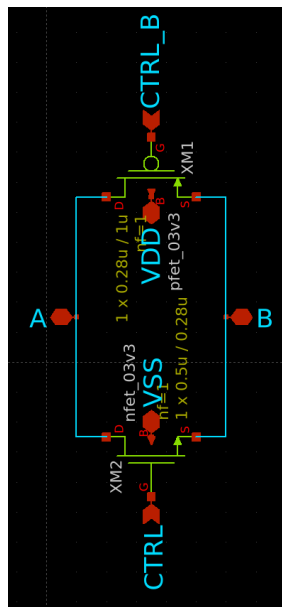
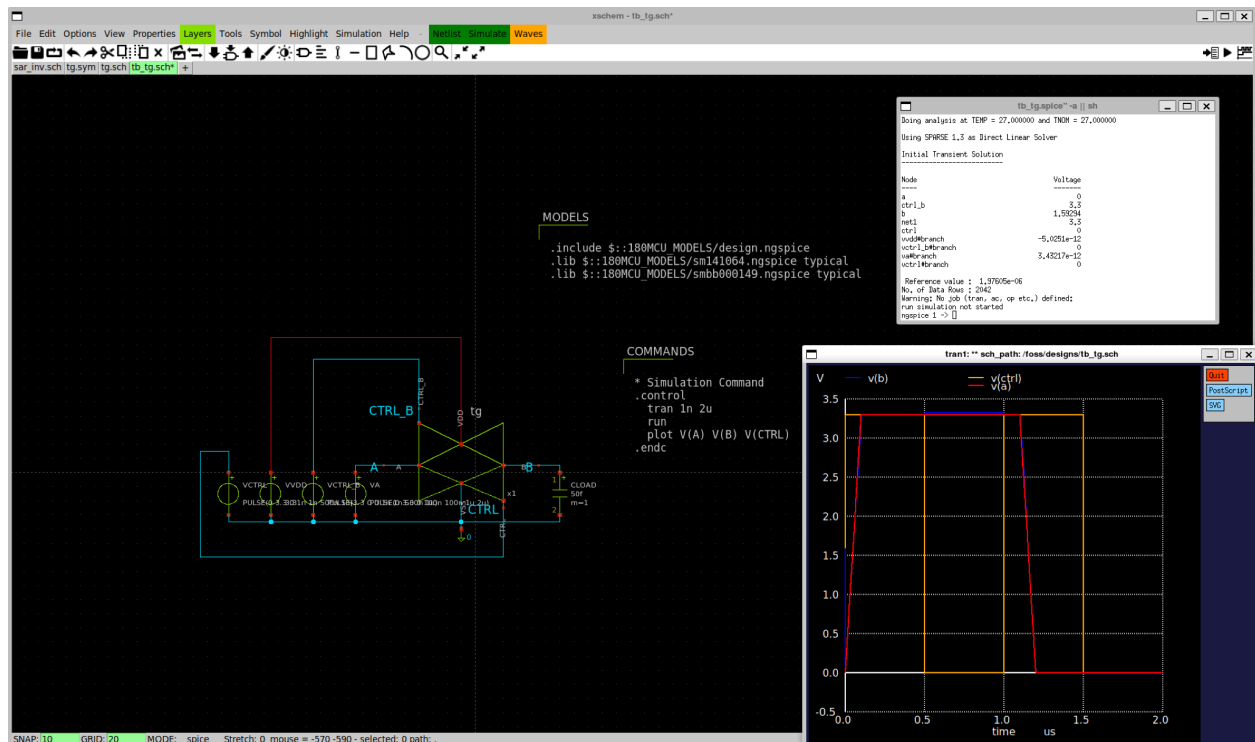
FINAL SAR TOP-LEVEL SCHEMATIC



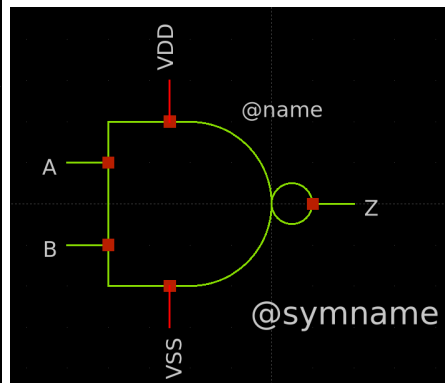
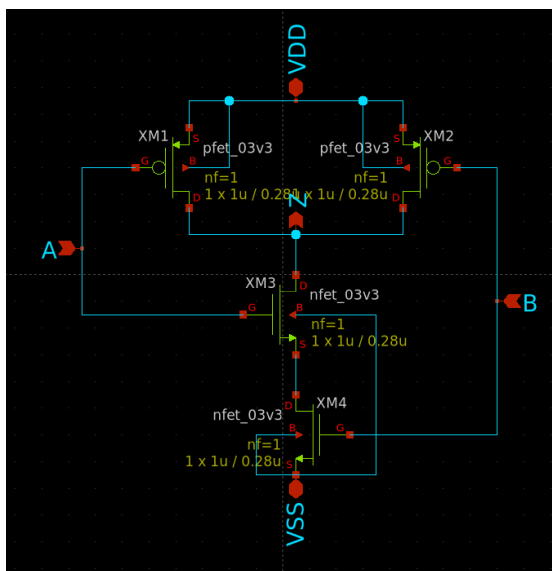
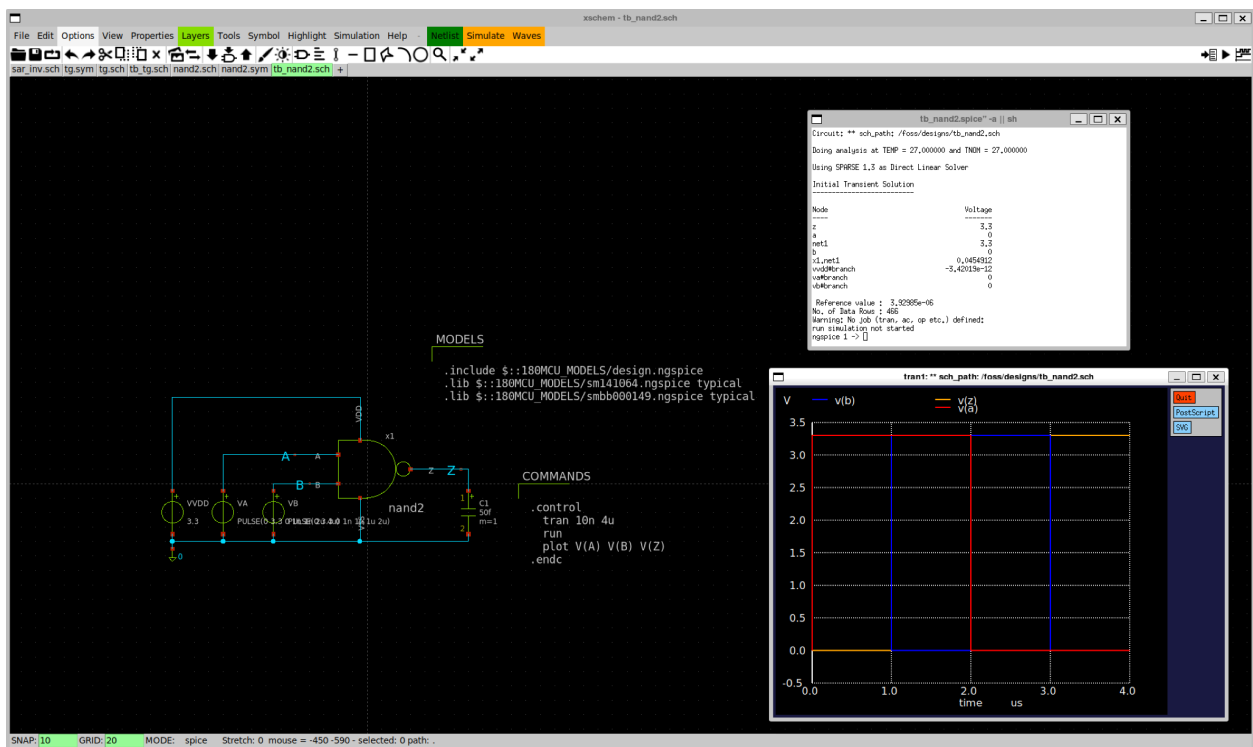
Inverter



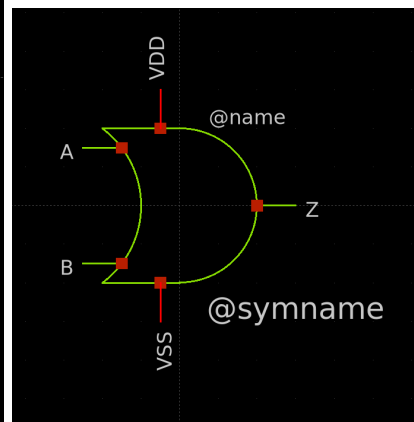
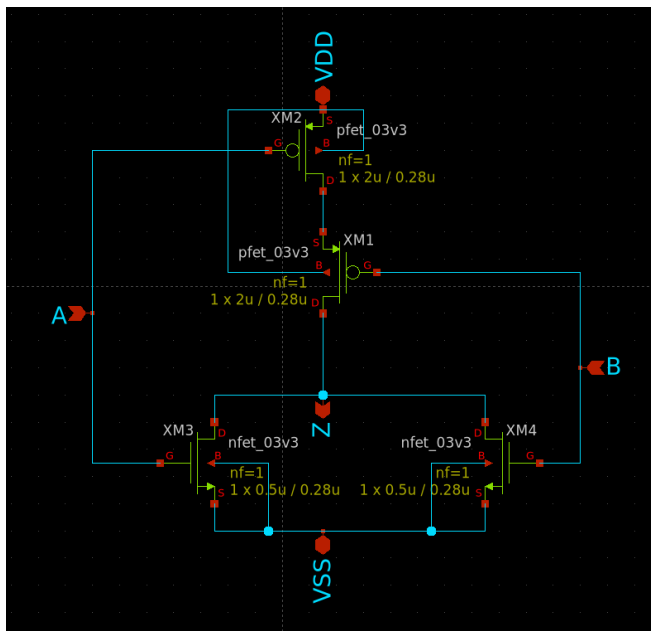
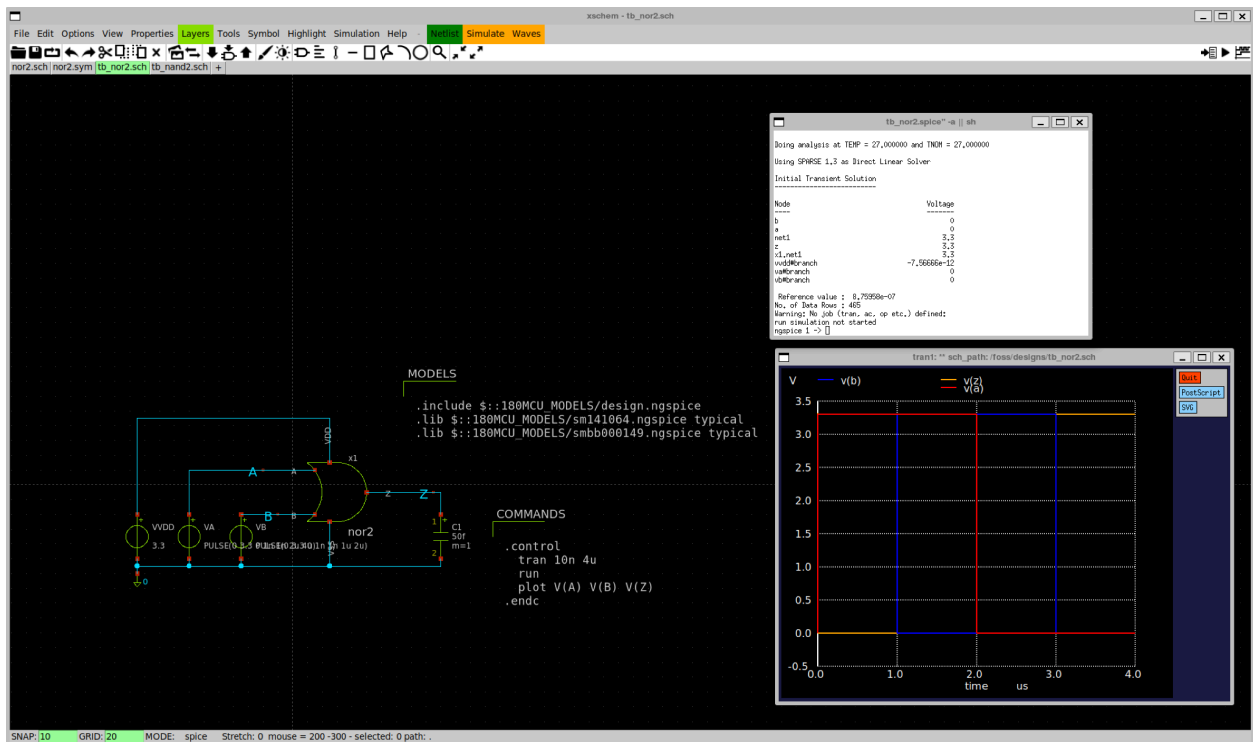
Transmission Gate



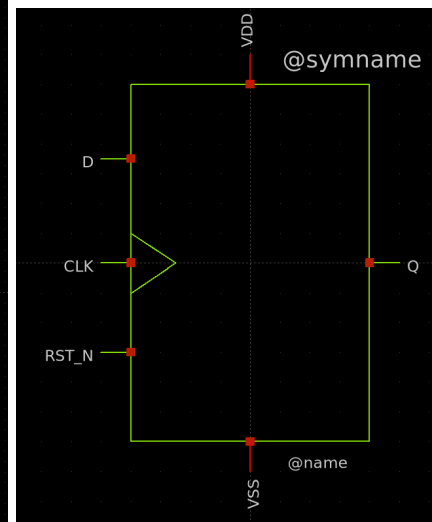
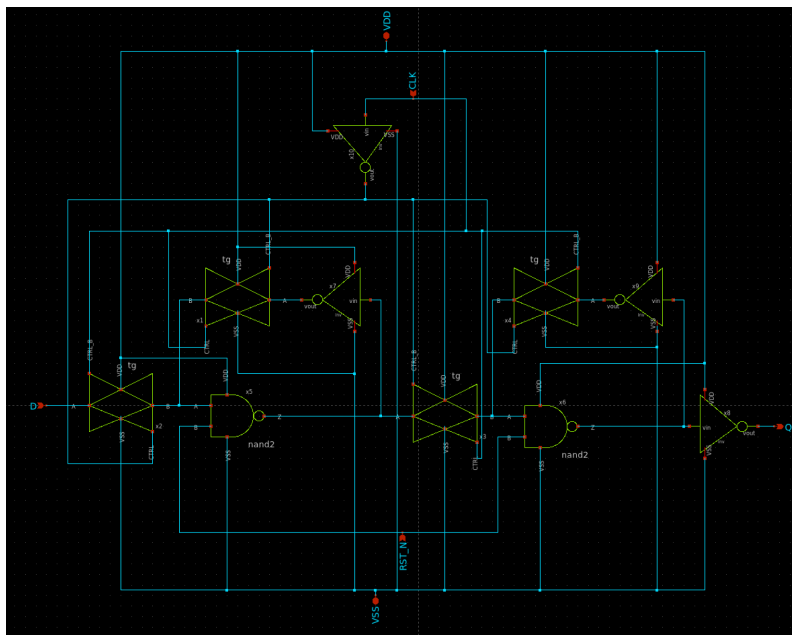
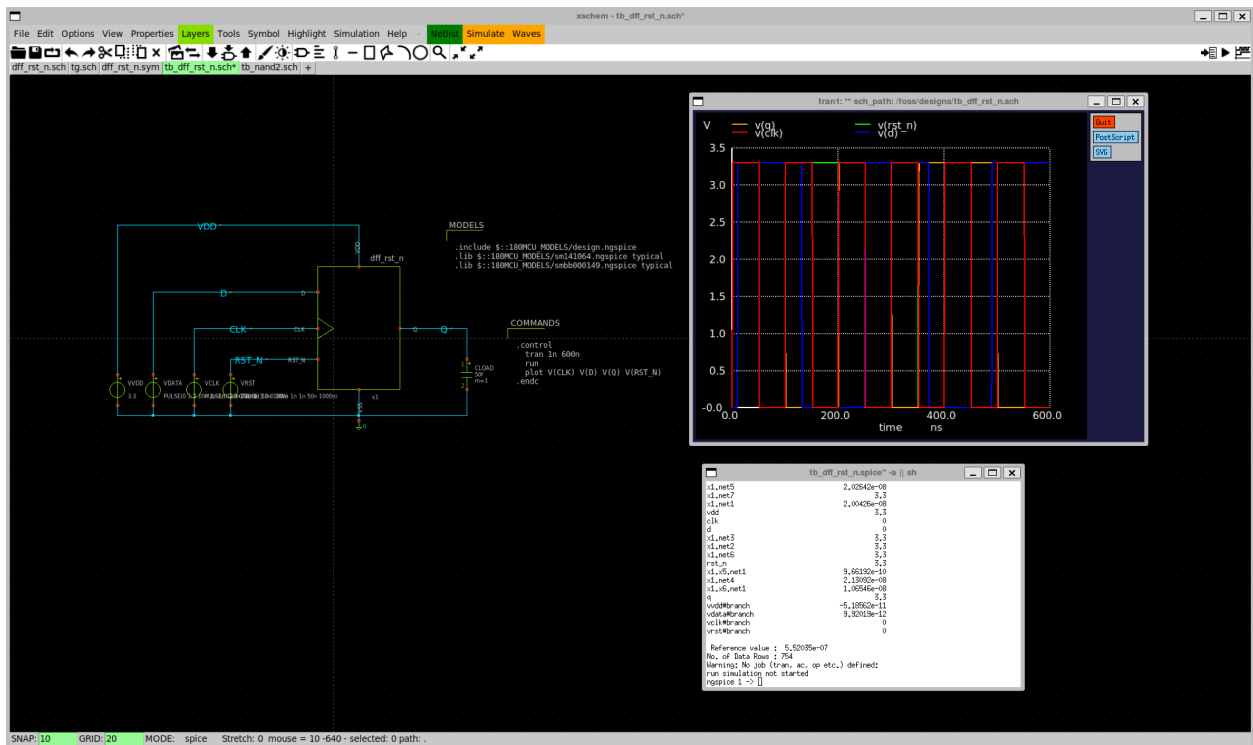
NAND2



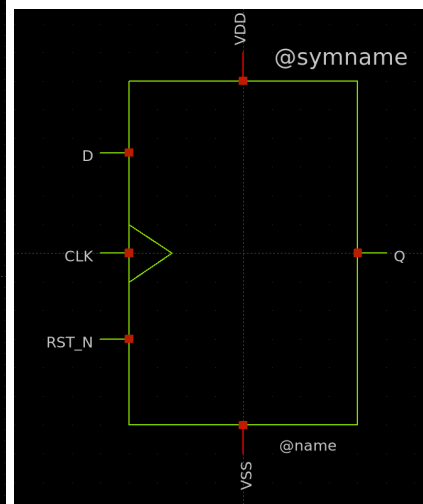
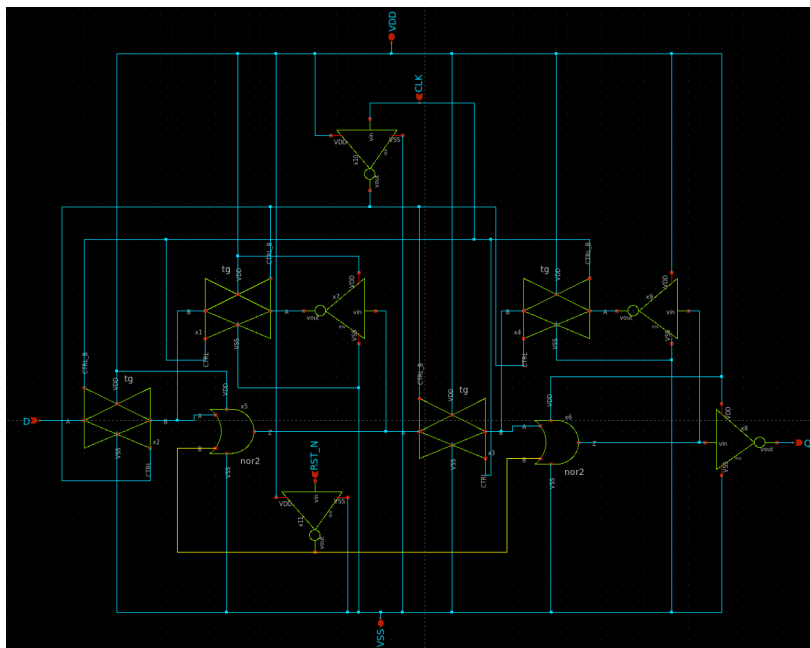
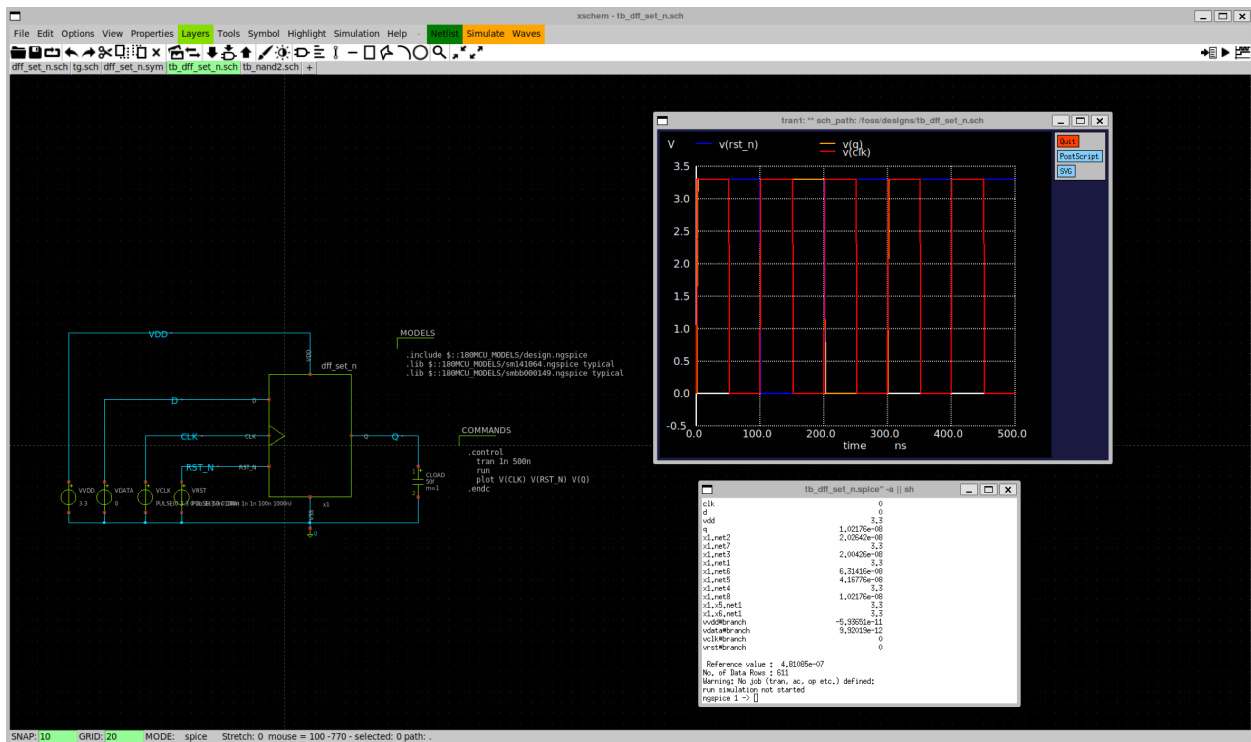
NOR2



DFF with Active-Low Asynchronous Reset



DFF with Active-Low Asynchronous Preset



(DOESNT WORK FIX NOR GATE/INVERTED RST_N HANDLING)